System Design Document

Enterprise Knowledge Assistant

1. Overview

The Enterprise Knowledge Assistant is a local-first Retrieval Augmented Generation (RAG) system built to help employees query internal documents using natural language. The solution is designed for enterprise environments where privacy, grounding, and reliability are essential. Instead of relying on external APIs, the application uses Ollama for both language generation and embeddings, combined with a local FAISS vector index for rapid retrieval.

The system enables users to ingest documents, search them semantically, and receive concise answers with source citations. It is suitable for HR policies, compliance documentation, technical guides, customer FAQs, and other internal knowledge sources.

---

2. Objectives

The primary goals of the system are:

1. Allow employees to ask natural-language questions about internal knowledge.
2. Retrieve relevant information from uploaded documents.
3. Generate grounded answers that remain faithful to the source content.
4. Provide source references so users can verify the result.
5. Run entirely locally for privacy and compliance reasons.

---

3. Business Context

Many organizations store important information across multiple documents and formats. Employees waste significant time manually searching these files for answers. The Enterprise Knowledge Assistant reduces this effort by offering a conversational interface that retrieves relevant information instantly and produces accurate answers grounded in the company’s own knowledge base.

This is especially valuable for:

- HR policies and employee support
- Technical documentation and troubleshooting
- Compliance and process documents
- Customer support knowledge bases

---

4. Functional Requirements

The system must support the following:

- Document ingestion from common file formats such as PDF, TXT, Markdown, DOCX, and CSV
- Text extraction and preprocessing
- Chunking and metadata preservation
- Vector embedding generation
- Semantic search over the indexed knowledge base
- Response generation using retrieved context
- Source citations in answers
- A user-facing interface through Streamlit and a REST API
- A command-line interface for usage and evaluation

---

5. High-Level Architecture

The system is composed of four major layers:

1. Interface Layer
- Streamlit web app for interactive use
- FastAPI API for programmatic access
- CLI for ingestion and evaluation

2. Application Layer
- Document ingestion workflow
- Retrieval pipeline
- Answer generation pipeline

3. Storage and Search Layer

- FAISS vector index for semantic retrieval
- Metadata JSON files for source and chunk information

4. Model Layer
- Ollama LLM for answer generation
- Ollama embedding model for document and query embeddings

The architecture is intentionally simple and modular so it can be extended in the future without redesigning the whole system.

---

6. Component Design

6.1 Ingestion Module

The ingestion module is responsible for loading files, extracting text, splitting documents into chunks, embedding those chunks, and storing them in a searchable index.

Responsibilities:

- detect supported file types,
- extract raw text from each file,
- split text into manageable chunks,
- attach metadata such as source filename and page number,
- generate embeddings,
- and store the chunks in the vector store.

This component ensures that the knowledge base remains structured and searchable.

6.2 Retrieval Module

The retrieval module takes a user question, converts it into an embedding, searches the index for relevant chunks, and returns the highest-scoring candidates.

The retrieval strategy uses:

- semantic similarity via embeddings,
- keyword matching using BM25-style scoring,
- and Reciprocal Rank Fusion (RRF) to blend both signals.

This increases retrieval quality by balancing meaning-based and exact-match retrieval.

6.3 Generation Module

The generation module uses the retrieved context as grounding to produce a concise answer. The prompt is designed to force the model to answer only from the provided context and include source references.

This design reduces hallucination risk and improves trustworthiness in enterprise settings.

6.4 Storage Module

The storage module uses FAISS for efficient similarity search. Each stored chunk contains:

- text content,
- source document information,
- page number,
- chunk index,
- and file hash metadata for deduplication.

FAISS is used because it is lightweight, fast, and avoids the dependency issues associated with some other vector database setups.

---

7. Data Flow

7.1 Ingestion Flow

1. User uploads or points the system to documents.
2. The ingestion module detects supported file types.
3. Text is extracted from the document.
4. The content is divided into chunks.
5. Embeddings are generated for each chunk.
6. Chunks are stored in the FAISS index with metadata.

7.2 Query Flow

1. The user submits a question.
2. The query is embedded using the same embedding model.

3. The system retrieves relevant chunks from the FAISS index.

4. The retrieved chunks are re-scored using keyword heuristics.

5. The most relevant context is assembled.

6. The LLM generates a grounded answer using that context.

7. The system returns the answer, source references, and confidence.

---

8. Technical Design Choices

8.1 Local-first deployment

The solution is designed to run entirely on the local machine. This helps avoid cloud data transfer and makes the system practical for private enterprise documents.

8.2 Ollama for LLM and embeddings

Ollama provides local model execution for both generation and embeddings. This ensures that no API keys are required and all processing remains on-device.

8.3 FAISS for vector search

FAISS provides efficient similarity search with low operational overhead. It is appropriate for medium-sized internal knowledge bases and is easy to run locally.

8.4 Prompt grounding

The system uses a strict system prompt instructing the model to answer only from the retrieved context. This is essential for minimizing hallucinations.

8.5 Source attribution

Each response includes document and page citations so the user can validate the generated answer.

---

9. Retrieval Strategy

The retrieval pipeline is intentionally hybrid rather than purely semantic.

Semantic retrieval

Semantic retrieval uses embeddings to find chunks that are conceptually similar to the user query.

Keyword retrieval

Keyword retrieval helps with exact-match questions, names, acronyms, or terminology that may not be captured well by embeddings alone.

RRF fusion

Reciprocal Rank Fusion combines the ranking lists from both retrieval methods to improve robustness. This is especially useful for enterprise documents where users may ask precise questions with domain-specific vocabulary.

---

10. Prompt Engineering

The generation prompt is designed to be strict and reliable:

- answer only from provided context,
- remain concise,
- cite the relevant source(s),
- and if no relevant information exists, explicitly say that the information is not available in the knowledge base.

This makes the assistant more dependable for practical enterprise use.

---

11. Handling Ambiguity and Uncertainty

The system is designed to avoid making up information. If the retrieved context does not contain enough evidence, the assistant returns a safe response stating that the knowledge base does not contain the requested information.

This is an important design principle for enterprise-grade AI systems because hallucinated answers can be harmful or misleading.

---

12. Evaluation Approach

The project includes an evaluation workflow to test the assistant on sample questions and measure its behavior.

Evaluation dimensions

- correctness of retrieved information,
- relevance of generated answers,
- citation behavior,
- and hallucination prevention.

Evaluation methodology

The assistant is tested against question-answer scenarios that include:

- direct factual questions,
- multi-step questions,
- cross-document lookups,
- and unanswered questions.

The evaluation process helps identify where retrieval quality or prompt behavior needs improvement.

---

13. Scalability Considerations

The current implementation is well-suited for small to medium internal knowledge bases. It can be scaled further in several ways:

- use a more scalable vector database such as Qdrant or Weaviate,
- add re-ranking for improved retrieval accuracy,
- parallelize ingestion for large document volumes,
- integrate asynchronous job queues for bulk ingestion,
- and add role-based access controls for multi-tenant enterprise usage.

The current system is intentionally simple and production-friendly while remaining extensible.

---

14. Security and Privacy Considerations

Because the system runs locally with Ollama, it offers strong privacy benefits for internal documents. Sensitive enterprise content does not need to leave the machine.

Potential future enhancements include:

- user authentication,
- role-based access control,
- audit trails,
- and document-level permissions.

---

15. Limitations

Although the system is effective for the assignment and practical demos, it has some limitations:

- ingestion can be slower for large document sets,
- FAISS does not support native deletion as robustly as some vector databases,
- scanned PDFs may require OCR support,
- and table-heavy documents may lose formatting fidelity during extraction.

These limitations are acceptable for the current implementation and can be addressed in future iterations.

---

16. Future Enhancements

Potential improvements include:

- cross-encoder reranking for retrieval precision,
- batch embedding for faster ingestion,
- Docker deployment for easier setup,
- conversation memory across turns,
- RBAC and authentication,
- and integration with a more advanced evaluation framework.

---

17. Conclusion

The Enterprise Knowledge Assistant demonstrates a strong end-to-end RAG implementation tailored for enterprise knowledge retrieval. It combines modern retrieval techniques, local LLM inference, and

practical software design to create a dependable assistant that can answer questions based on internal documentation while maintaining traceability and privacy.